OAK RIDGE National Laboratory

# QRMI and QDMI

## What the comparison found

*Sixteen axes · one test vehicle · live runs on the ORNL IQM 20-qubit system*

Doug Oucharek · openQSE · August 2026

ORNL is managed by UT-Battelle LLC for the US Department of Energy

U.S. DEPARTMENT OF ENERGY

OAK RIDGE National Laboratory

THE SETUP

# One test vehicle, three arms, sixteen axes

- Both interfaces behind one QFw contract
- A native-client arm as the control
- Sixteen axes, every entry cited
- Live runs on the ORNL q20, wide-area path

## Pinned versions

| | |
|---|---|
| qrmi | **0.17.2** |
| iqm-qdmi (QDMI on IQM) | **1.2.0** |
| mqt-core (FoMaC) | **3.7.0** |
| iqm-client | **34.0.1** |

*Findings hold at these versions.*

***Every claim ahead is cited or measured.***

THE BASELINE

# Both work, and they agree

## 20 qubits, 30 edges

Same topology through both

## Identical counts

One circuit, both paths

## One shared record

qhw-result-v1 from each

***The question is what each one shows you, and what it hides.***

READING THE FINDINGS

# Two kinds of findings

DIFFERENCE

## The libraries diverge

*Design choices, not defects*

SHARED SHORTCOMING

## Neither library provides it

*Where a specification earns its keep*

***Differences explain the libraries. Shortcomings define the spec work.***

FINDING 1

DIFFERENCE

# Different layers

## QRMI · unit is a resource

- One trait, twelve methods, every backend
- Vendors live inside the QRMI tree
- Deploys with the scheduler

*Faces resource managers and middleware*

## QDMI · unit is a device

- A client API and a device ABI
- Vendors ship a shared library
- Typed properties, typed NOTSUPPORTED

*Faces device providers and tools*

***One sits above the other. Most differences follow from that.***

FINDING 2

SHARED SHORTCOMING

# Capability not discoverable

## QRMI

***Implied by the resource type***

- Seven types, four payload shapes
- The mapping lives in the caller

## QDMI

***Queried per device***

- Program formats from a defined list
- Typed NOTSUPPORTED for what is absent

### What QFw had to do

The shim routes every call from a hand-kept map. It drifted, silently, until a routing assertion caught it.

***Neither can say which calls will work for a resource.***

FINDING 3

SHARED SHORTCOMING

# No working hold

## What acquire() actually does

**IBM Qiskit Runtime**

Real session work at the provider

REAL

**IBM Direct Access**

A generated id, nothing else

NO-OP

**IQM Server**

A generated id, nothing else

NO-OP

*All three return success · qrmi 0.17.2*

– release() returns success too, without touching the device

– QDMI has no acquire, reserve, or lock at all

– Even a real hold would have nothing to decide on

FINDING 4

SHARED SHORTCOMING

# Submitted to, not scheduled

**The site orders the work**

*fair share, priority, reservations*



**The provider orders it again**

*policy the site cannot see or set*

**no channel between them**

***The two queue problem, long familiar outside quantum.***

- No priority, no deadline, no reservation reference
- Queue position goes to a log line, not to the caller
- No timing, so reordering is invisible

***A high-priority job has no way to say so.***

FINDING 5

SHARED SHORTCOMING

# Provider data discarded

**34.9 ms**

queue wait, provider-reported

**107.3 ms**

execution, provider-reported

**3.99 s**

what the caller sees, undivided

*One run. The provider's own total was 542 ms. Neither interface passes any of it on.*

- QDMI keeps only the metrics it models
- Raw but provider-shaped on the QRMI path
- No record of what was sent, or under which calibration

***Lost above the provider, not missing at it.***

FINDING 6

DIFFERENCE

# Observation costs

| | QRMI | QDMI on IQM | native control |
|---|---|---|---|
| Cold first look | 1334.6 ms | 3376.8 ms | 2831.1 ms |
| Warm repeat | 11.8 ms | 13.1 ms | 512.6 ms |
| Connections | 1, pooled | 5, fresh TLS | no reuse |

- Both interfaces cache. Roughly forty times cheaper warm.
- The cold gap is transport handling, not interface design
- Measured over a wide-area path, on purpose

FINDING 7

DIFFERENCE

# Failures in disguise

| QDMI | QRMI introspection |
| --- | --- |
| A typed code at the failing call | Null fields inside a success, and no logger |

**Observed on hardware · the QRMI path**

tunnel drops → fetches fail silently → null target cached → surfaces two layers up as bad device data

ALSO FOUND

# Five more, one line each

- IQM JSON means a run request in QRMI, a circuit in QDMI — DIFFERS
- Caller-side assembly broke on a private SDK helper — DIFFERS
- QDMI grows without breaking. QRMI breaks to grow. — DIFFERS
- Same bearer token, different injection — DIFFERS
- Placement is never a field, only names inside the program — SHARED

***Each has a full section in the document.***

# Discussion

github.com/openQSE/development-analysis · qrmi-qdmi

# Where each finding lives in the document

| | | |
|---|---|---|
| 1 Different layers | DIFFERS | Interface Role And Scope |
| 2 Capability not discoverable | SHARED | Device Discovery · Extensibility And Versioning |
| 3 No working hold | SHARED | Admission |
| 4 Submitted to, not scheduled | SHARED | Device Scheduler Control · Telemetry |
| 5 Provider data discarded | SHARED | Telemetry · Calibration And Quality Data · Job Lifecycle And Results |
| 6 Observation costs | DIFFERS | Telemetry, the measured entries |
| 7 Failures in disguise | DIFFERS | Error Model · the note under Job Lifecycle And Results |

*Measurement method and raw data live in the QFw repo, examples/measure_shim_introspection.py and examples/measurements/.*

# Measurement notes

- Five samples, medians. Cold ranges 1238 to 1500 and 3193 to 3397 ms.
- Wide-area path, SSH tunnel to the q20
- Warm means a repeat query in the same process, no network
- The native client opened 25 connections where the interfaces opened none
- The 2.5x gap tracks connection reuse, one pooled against five fresh